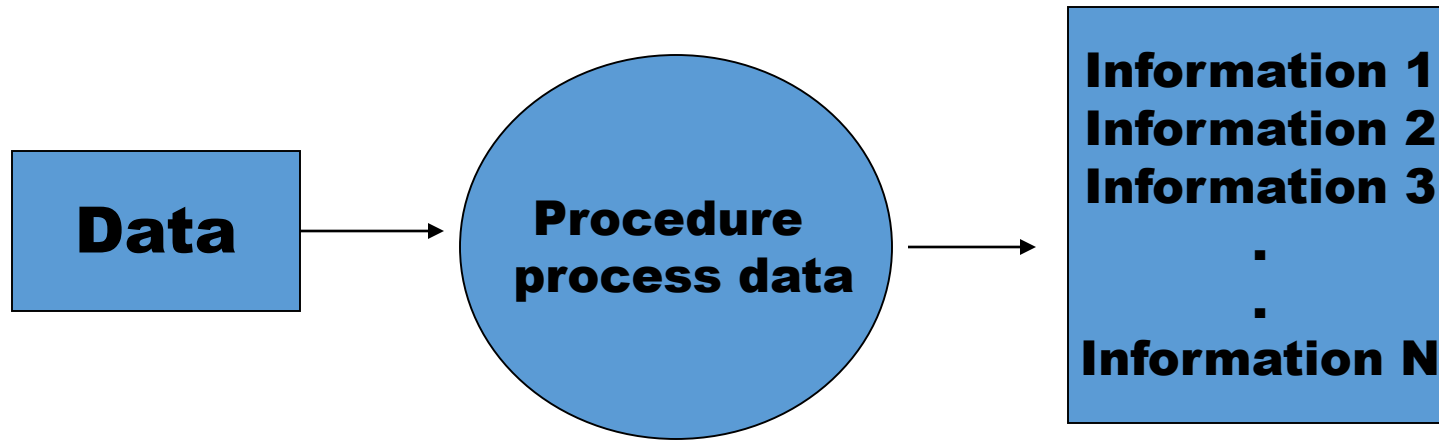


DATA STRUCTURE

What is Data?

- Data is a collection of raw, un-organised facts and details like text, observations, figures, symbols and description of things etc. In other words, data does not carry any specific purpose and has no significance by itself. Moreover, data is measured in terms of bits and bytes – which are basic units of information in the context of computer storage and processing.
- **What is Information?**

Information is the processed, organized and structured data. It provides context for data and enables decision making. For example, a single customer's sale at a restaurant is data – this becomes information when the business is able to identify the most popular or least popular dish.



when data is processed by applying certain rules, new generated data becomes information. This new generated data (information) conveys some meaning and hence can be used for decision making.

For Example:

Data

34

24/01/1980

ISBN-569-908-4765

Information

No. of employee

DOB of a employee

Book Number

Difference Between Data and Information

Data	Information
Data is unorganized and unrefined facts	Information comprises processed, organised data presented in a meaningful context
Data is an individual unit that contains raw materials which do not carry any specific meaning.	Information is a group of data that collectively carries a logical meaning.
Data doesn't depend on information.	Information depends on data.
It is measured in bits and bytes.	Information is measured in meaningful units like time, quantity, etc.
Raw data alone is insufficient for decision making	Information is sufficient for decision making
An example of data is a student's test score	The average score of a class is the information derived from the given data.

Need of Data Structures

As applications are getting complex and amount of data is increasing day by day, there may arise the following problems:

- **Data Search** – Consider an inventory of 1 million(10^6) items of a store. If the application is to search an item,
- **Processor speed** – Processor speed although being very high, falls limited if the data grows to billion records.
- **Multiple requests** – As thousands of users can search data simultaneously on a web server, even the fast server fails while searching the data.

Data Structure:

The logical or mathematical model of a particular organization of data is called ***data structure***.

A particular data model depends on two consideration:

1. It must be rich enough in structure to reflect the existing relationship between the data.
2. The structure should be simple enough that one can effectively process that data when necessary.

In other words data structure is a collection of data elements whose organization is characterized by accessing operations that are used to store and retrieve the individual data element.

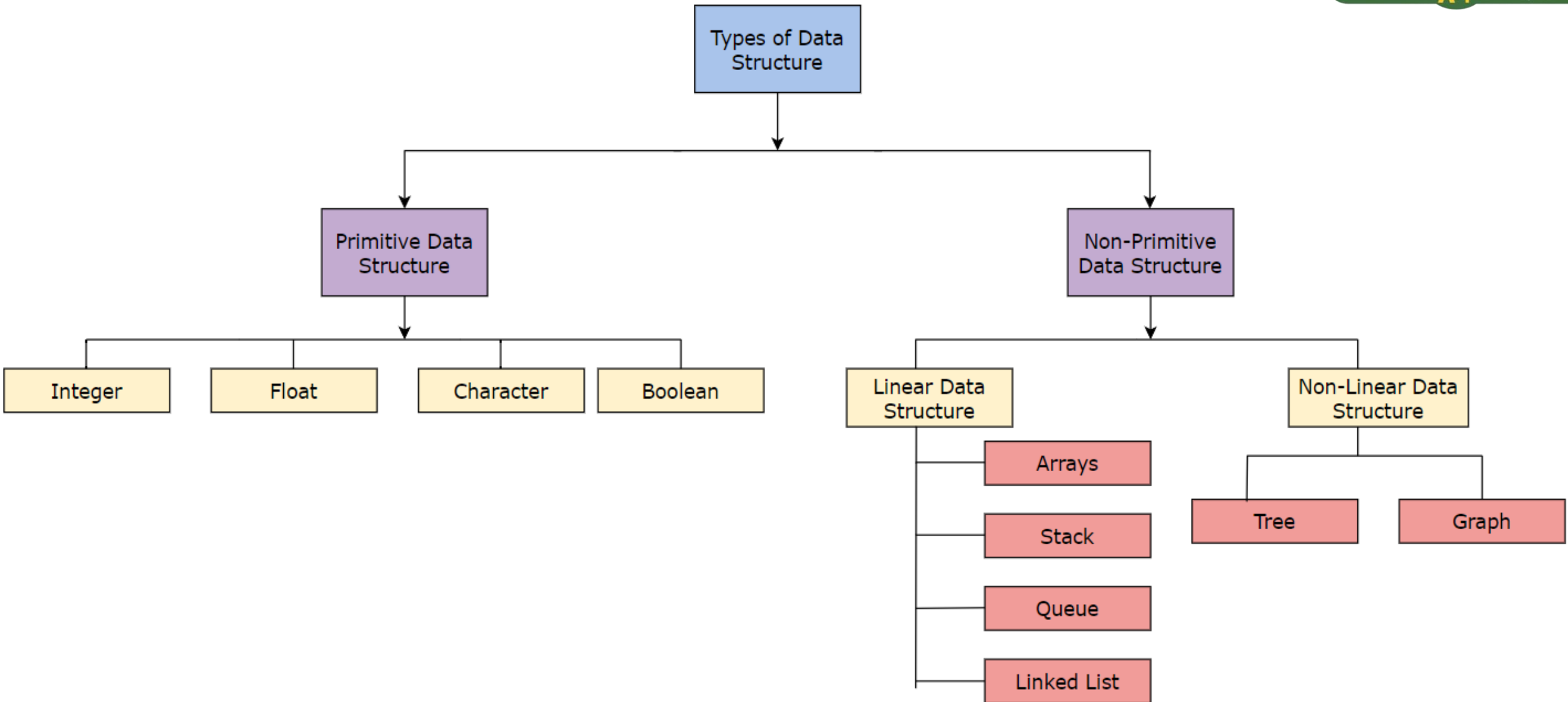
Operation

- **Traversing:** Traversing a Data Structure means to visit the element stored in it. This can be done with any type of Data Structure.
- **Insertion:** Insertion can be defined as the process of adding the elements to the data structure at any location.
- **Deletion:** The process of removing an element from the data structure is called Deletion. We can delete an element from the data structure at any random location.

Continue...

- **Searching:** The process of finding the location of an element within the data structure is called Searching. There are two algorithms to perform searching, Linear Search and Binary Search.
- **Sorting:** The process of arranging the data structure in a specific order is known as Sorting. There are many algorithms that can be used to perform sorting, for example, insertion sort, selection sort, bubble sort
- **Merging:** Combine two similar type of Data structure

- Types of data structure:



Primitive data structure

- Primitive data structure is a data structure that can hold a single value in a specific location.
- The examples of primitive data structure are float, character, integer and pointer.
- The value to the primitive data structure is provided by the programmer. There are the four primitive data structures: float, character, integer and pointer.

Non-primitive data structure

- The non-primitive data structure is a kind of data structure that can hold multiple values either in a contiguous or random location.
- The non-primitive data types are defined by the programmer. The non-primitive data structure is further classified into two categories, i.e., linear and non-linear data structure.

linear data structure

- A **linear data structure** has data elements connected to each other so that elements are arranged in a sequential manner and each element is connected to the element in front of it and behind it. This way, the structure can be traversed in a single run
- Array
- Stack
- Queue
- Linked list.

Non-linear data structure

- in this form of data structure, the data elements don't stay arranged linearly or sequentially.
- a user can't traverse all of its elements in a single run.
- Tree
- Graph

S.NO Linear Data Structure

Non-linear Data Structure

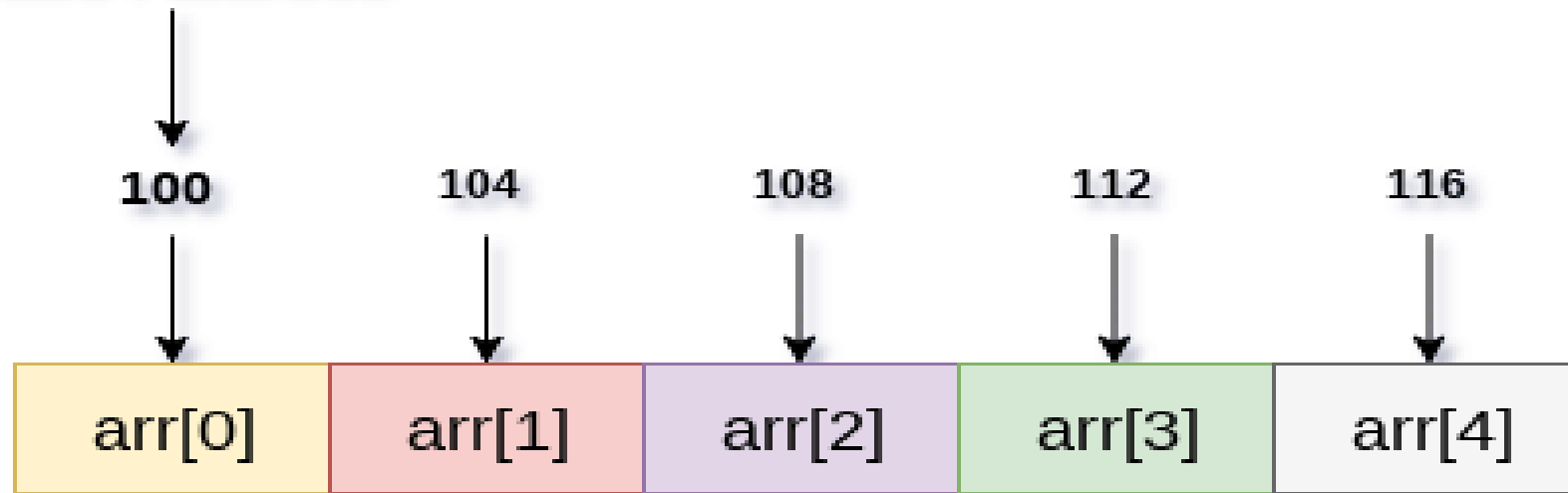
- | | | |
|----|--|--|
| 1. | In a linear data structure, data elements are arranged in a linear order where each and every elements are attached to its previous and next adjacent. | In a non-linear data structure, data elements are attached in hierarchically manner. |
| 2. | In linear data structure, single level is involved. | Whereas in non-linear data structure, multiple levels are involved. |
| 3. | Its implementation is easy in comparison to non-linear data structure. | While its implementation is complex in comparison to linear data structure. |

- | | |
|---|--|
| 4. In linear data structure, data elements can be traversed in a single run only. | While in non-linear data structure, data elements can't be traversed in a single run only. |
| 5. In a linear data structure, memory is not utilized in an efficient way. | While in a non-linear data structure, memory is utilized in an efficient way. |
| 6. Its examples are: array, stack, queue, linked list, etc. | While its examples are: trees and graphs. |

Array

- Arrays are defined as the collection of similar type of data items stored at contiguous memory locations.
- Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double, float, etc.
- Array is the simplest data structure where each data element can be randomly accessed by using its index number.

Base Address

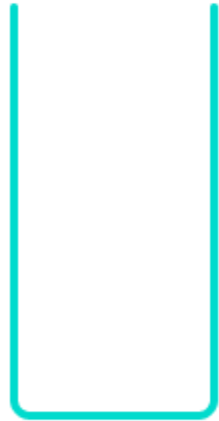


`int arr[5]`

Stack:

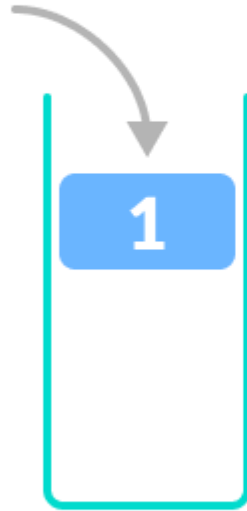
- Stack is a **linear data structure which follows a particular order in which the operations are performed.**
- The order may be LIFO(Last In First Out) or FILO(First In Last Out).
- There are many real-life examples of a stack. Consider an example of plates stacked over one another in the canteen.

TOP = -1



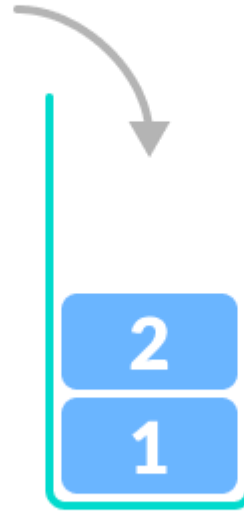
**empty
stack**

**TOP = 0
stack[0] = 1**



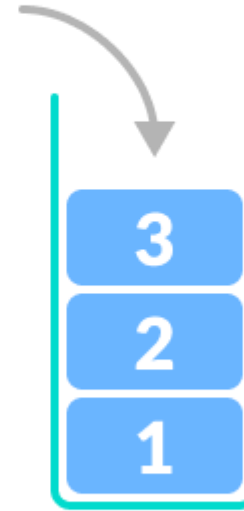
push

**TOP = 1
stack[1] = 2**



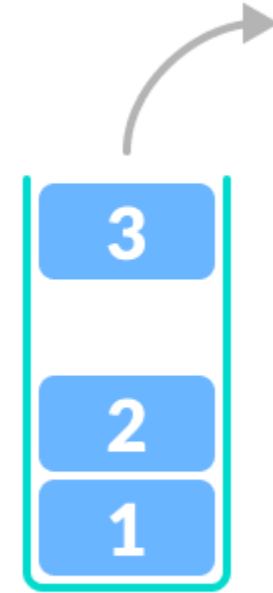
push

**TOP = 2
stack[2] = 3**



push

**TOP = 1
return stack[2]**

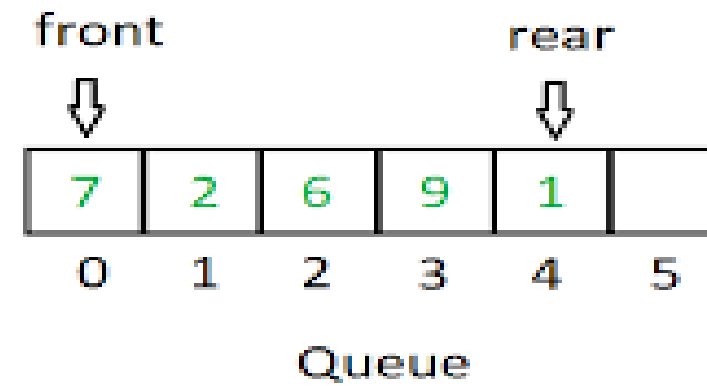
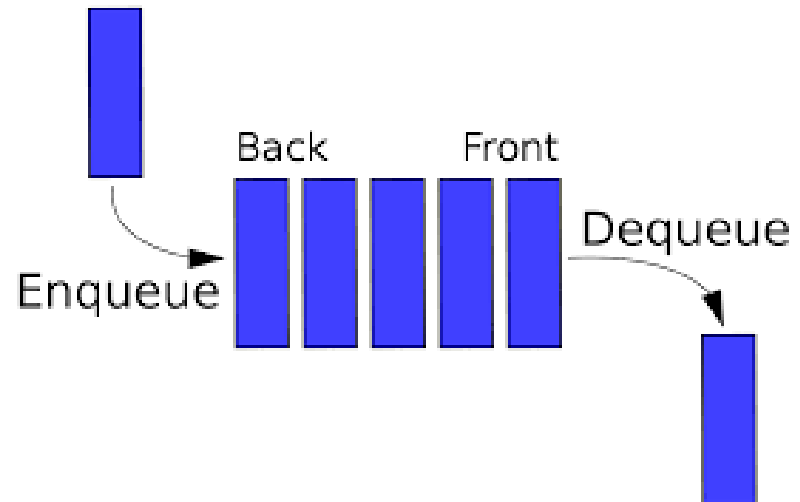


pop

Queue:

- ▶ Queue is also a linear data structure which works on FIFO order. So that First In First Out .
- ▶ In queue element is always added at rear of queue and removed from front of queue.
- ▶ Queue applications are in CPU scheduling, Disk Scheduling, IO Buffers, pipes, file IO.

Example

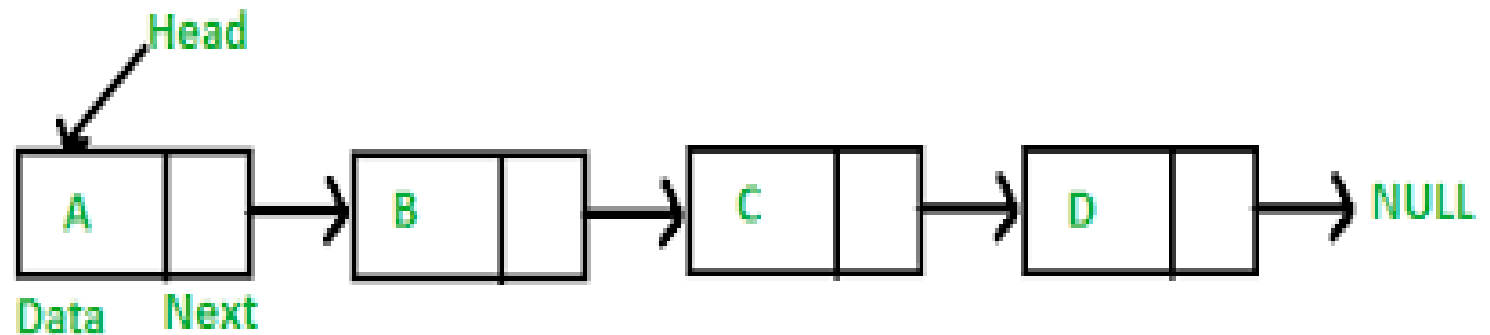


linked list :

- ▶ a **linked list** is a linear collection of data elements, in which linear order is not given by their physical placement in memory.
- ▶ Elements may be added in front, end of list as well as middle of list.
- ▶ Linked List may be use for dyanamic implementation of stack and queue.

A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers.

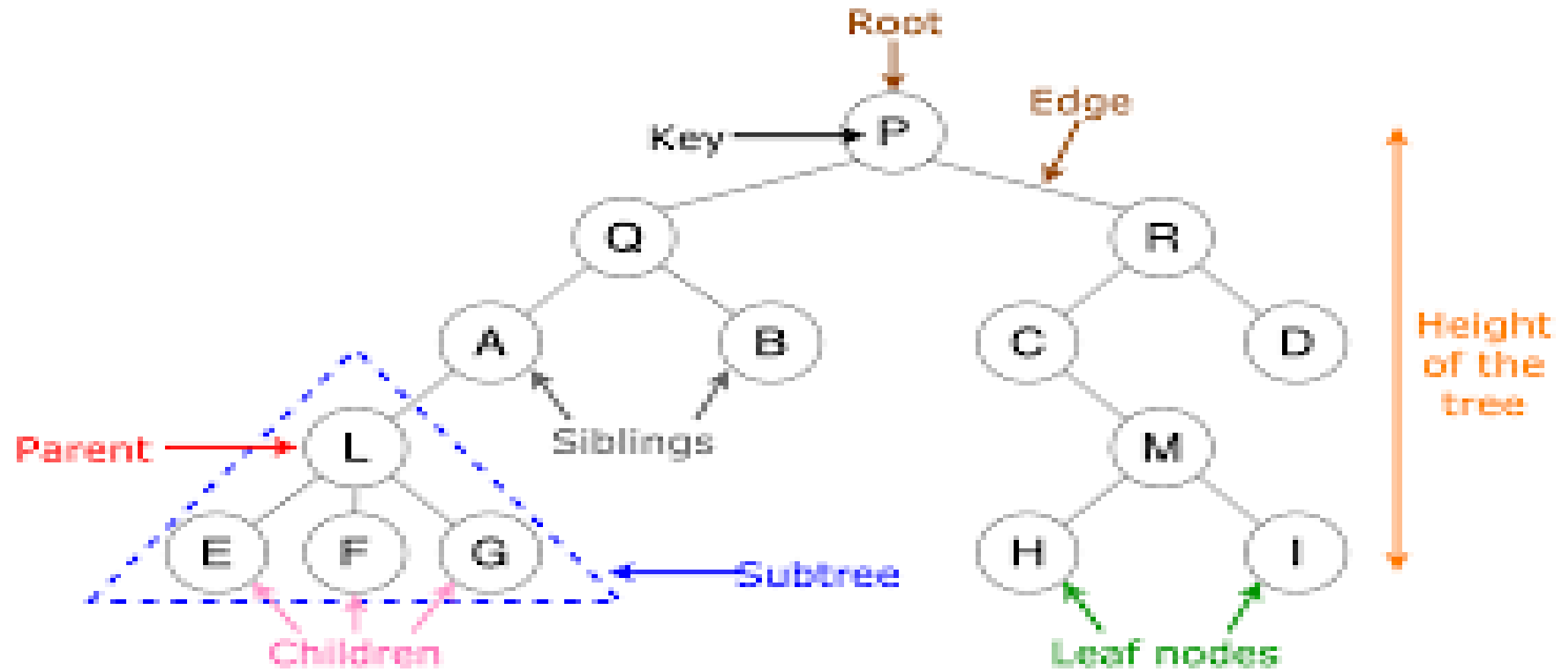
Linked lists can be of multiple types: singly, doubly, and circular linked list.



Tree:

- ▶ A tree is a non linear data structure. a root value and subtrees of children with a parent node, represented as a set of linked nodes.
- ▶ Nodes can be added at any different node.
- ▶ Tree applications includes:–
- ▶ Manipulate hierarchical data.
 2. Make information easy to search (see tree traversal).
 3. Manipulate sorted lists of data.
 4. As a workflow for compositing digital images for visual effects.
 5. Router algorithms

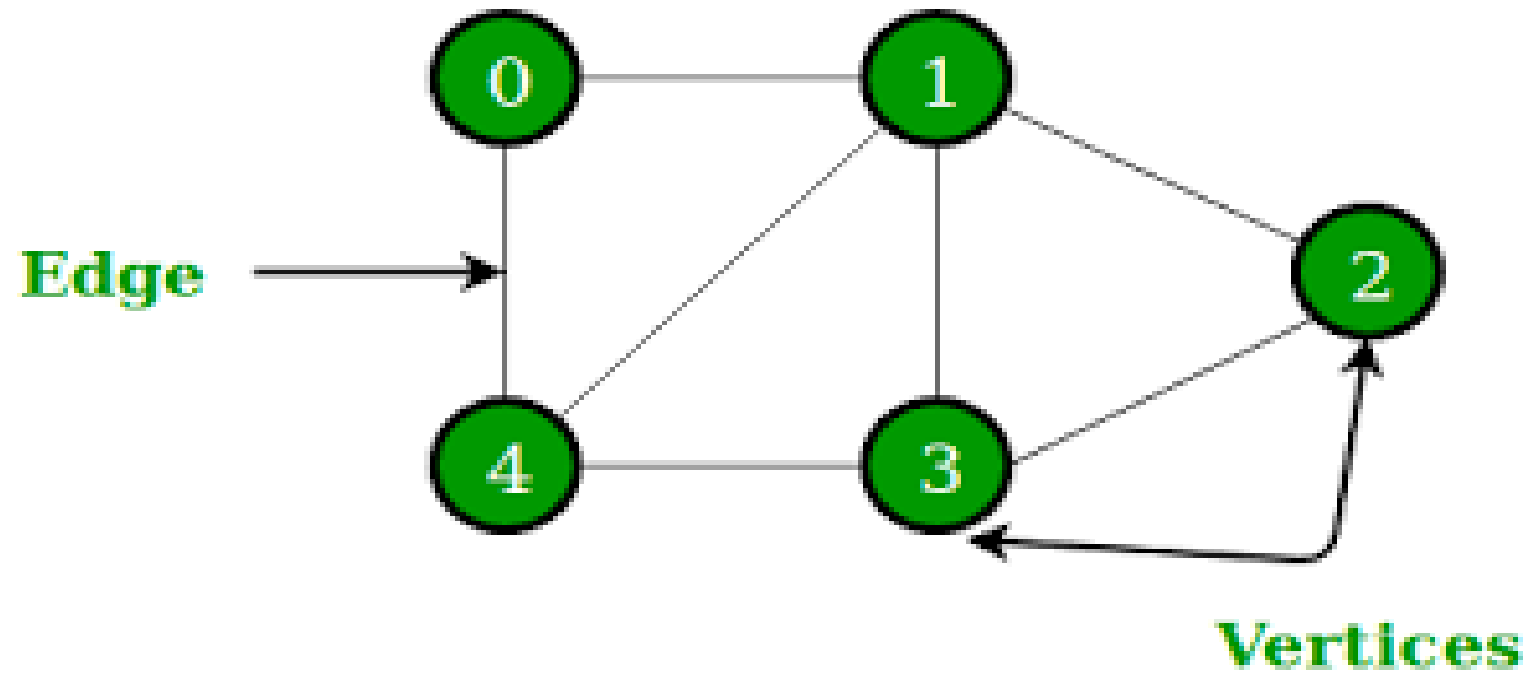
Example:



Graph:

- ▶ A graph is a non linear data structure. A set of items connected by edges. Each item is called a vertex or node. Formally, a **graph** is a set of vertices and a binary relation between vertices, adjacency.
- ▶ Graph applications:–
- ▶ finding shortest routes, searching, social network connections, internet routing.

Example:



What is an algorithm

An algorithm is an effective, efficient and best method which can be used to express solution of any problem within a finite amount of space and time and in a well-defined formal language.

In other words, we can say that,

1. Step by step procedure for solving any problem is known as algorithm.
2. An algorithm is a finite set of instructions that, if followed, accomplishes a particular task.
3. An algorithm is a sequence of computational steps that transform the input into a valuable or required output.
4. Any special method of solving a certain kind of problem is known as algorithm.

All Algorithms must satisfy the following criteria -

1) Input

There are more quantities that are extremely supplied.

2) Output

At least one quantity is produced.

3) Definiteness

Each instruction of the algorithm should be clear.

4) Finiteness

The process should be terminated after a finite number of steps.

5) Effectiveness

Every instruction must be basic enough to be carried out theoretically or by using paper and pencil.

Properties of Algorithm

- **Non Ambiguity**

Each step in an algorithm should be non-ambiguous. That means each instruction should be clear and precise. The instruction in any algorithm should not denote any conflicting meaning. This property also indicates the effectiveness of algorithm.

- **Range of Input**

The range of input should be specified. This is because normally the algorithm is input driven and if the range of input is not being specified then algorithm can go in an infinite state.

- **Multiplicity**

The same algorithm can be represented into several different ways. That means we can write in simple English the sequence of instruction or we can write it in form of pseudo code. Similarly, for solving the same problem we can write several different algorithms.

- **Speed**

The algorithms written using some specified ideas. But such algorithm should be efficient and should produce the output with fast speed.

- **Finiteness**

The algorithm should be finite. That means after performing required operations it should be terminate.

Simple algorithm for calculating the "Multiplication" of two numbers.

- Step 1: Start.
- Step 2: declare two variables (a & b).
- Step 3: Store integer values in 'a' and 'b.' -> Input
- Step 4: declare a variable named 'mul'
- Step 5: Store the mul of 'a' and 'b' in a variable named 'mul' -> Output
- Step 6: End.

Example

Let's try to learn algorithm-writing by using an example.

Problem – Design an algorithm to add two numbers and display the result.

Step 1 – START

Step 2 – declare three integers a, b & c

Step 3 – define values of a & b

Step 4 – add values of a & b

Step 5 – store output of step 4 to c

Step 6 – print c

Step 7 – STOP

Algorithm Analysis:

Analysis of algorithms is the determination of the amount of time and space resources required to execute it.

Why Analysis of Algorithms is important?

- To predict the behavior of an algorithm without implementing it on a specific computer.
- It is much more convenient to have simple measures for the efficiency of an algorithm than to implement the algorithm and test the efficiency every time a certain parameter in the underlying computer system changes.
- It is impossible to predict the exact behavior of an algorithm. There are too many influencing factors.
- The analysis is thus only an approximation; it is not perfect.
- More importantly, by analyzing different algorithms, we can compare them to determine the best one for our purpose.

QUALITIES OF GOOD ALGORITHM:

The factors that we need to consider while determining the quality of a good Algorithm are:

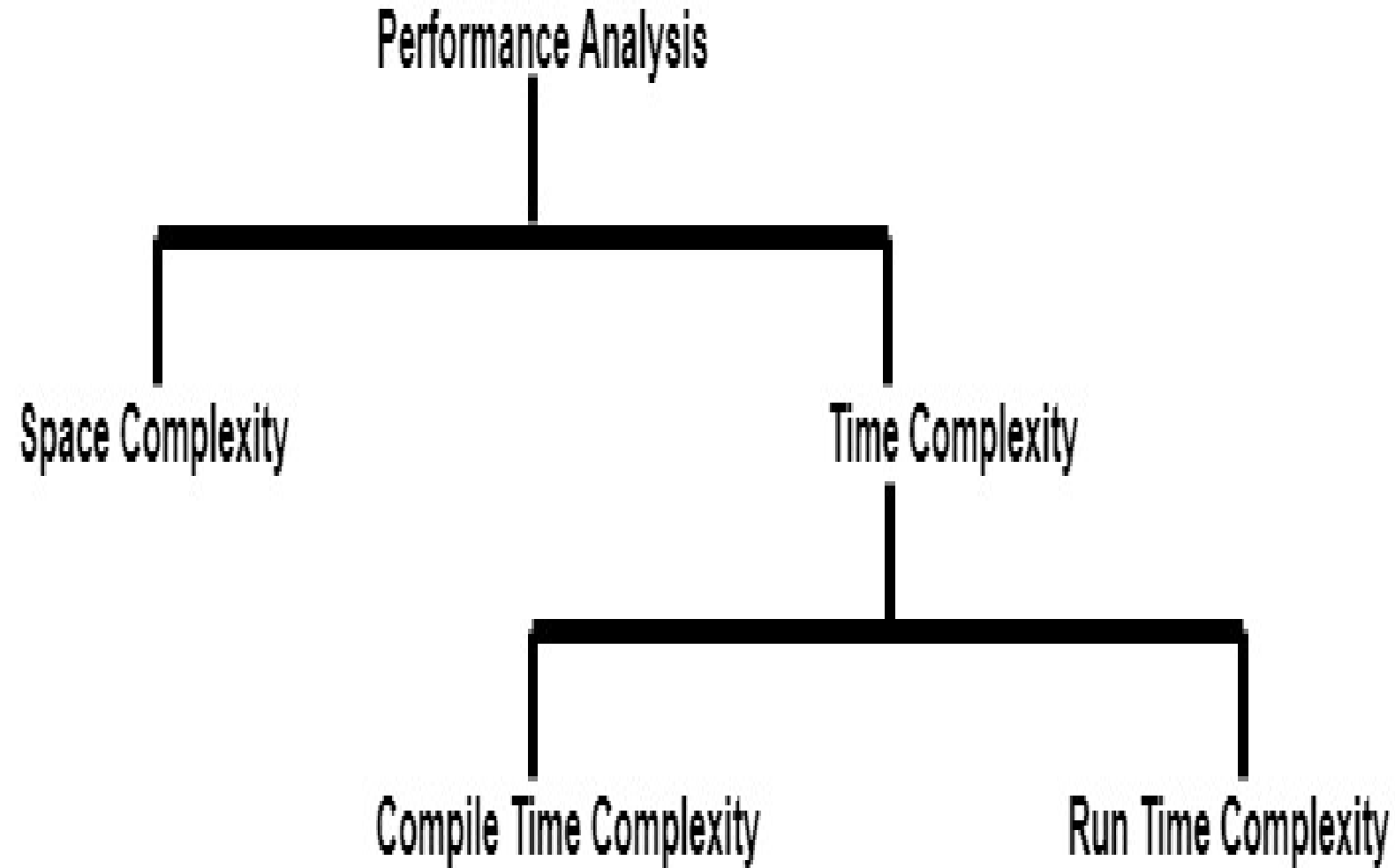
- **Time:** The amount of time it takes to run as a function of the length of the input is known as time requirement.
- **Memory:** The amount of memory utilized by the algorithm (including the inputs) to execute and create the output is referred to its memory requirement.
- **Accuracy:** There can be more than one solution to the given problem, but the one which is the most optimal is termed as accurate.

Analyzing parameters

- Space Complexity
- Time Complexity



Categorization of Complexity:



What Is Space Complexity?

The amount of memory used by a program to execute it is represented by its space complexity (Main memory space).

- *Space Complexity of an algorithm is total space taken by the algorithm to completion of its execution with respect to the input size.*

Space Complexity

- The space complexity of an algorithm is calculated by determining the following 2 components:
- **Fixed Part:**
- **Variable Part:**

Space Complexity

- **Fixed Part:**
- This refers to the space that is required by the algorithm.
- For example, input variables, output variables, program size, etc.

Space Complexity

- Variable Part:
- This refers to the space that can be different based on the implementation of the algorithm.
- For example, temporary variables, dynamic memory allocation, recursion stack space, etc.
- •
- Therefore Space complexity $S(P)$ of any algorithm P is $S(P) = C + SP(I)$, where C is the fixed part and $S(I)$ is the variable part of the algorithm, which depends on instance characteristic I .

Space Complexity

- **Example:** Consider the below algorithm for Linear Search
- *Step 1: START*
Step 2: Get n elements of the array in arr and the number to be searched in x
Step 3: Start from the leftmost element of arr[] and one by one compare x with each element of arr[]
Step 4: If x matches with an element, Print True.
Step 5: If x doesn't match with any of the elements, Print False.
Step 6: END

Space Complexity

- *Here, There are 2 variables $arr[]$, and x , where the $arr[]$ is the variable part of n elements and x is the fixed part.*
- *Hence $S(P) = 1+n$. So, the space complexity depends on n (number of elements).*
- *Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.*

What Is Time Complexity?

- Time complexity (CPU Time) is a type of computational complexity that describes the time required to execute an algorithm. The time complexity of an algorithm is the amount of time it takes for each statement to complete. As a result, it is highly dependent on the size of the processed data.

Time Complexity

- **Time Complexity:**
- The time complexity of an algorithm refers to the amount of time required by the algorithm to execute and get the result.
- This can be for normal operations, conditional if-else statements, loop statements, etc.

Time Complexity

- **How to Calculate, Time Complexity?**

The time complexity of an algorithm is also calculated by determining the following 2 components:

- **Constant time part**
- **Variable Time Part**

Time Complexity

- **Constant time part:**
 - Any instruction that is executed just once comes in this part.
 - For example, input, output, if-else, switch, arithmetic operations, etc.
- **Variable Time Part:**
 - Any instruction that is executed more than once, say n times, comes in this part.
 - For example, loops, recursion, etc.

Time Complexity

- Therefore Time complexity $T(P)$
- $T(P)$ of any algorithm P is **$T(P) = C + TP(I)$** ,
- where C is the constant time part and $TP(I)$ is the variable part of the algorithm, which depends on the instance characteristic I .

Time Complexity

- Best, Worst, and Average Case Complexity

Time Complexity

- Best, Worst, and Average Case Complexity:
- In analyzing algorithms, we consider three types of time complexity:
- **Best-case complexity ($O(\text{best})$):** This represents the minimum time required for an algorithm to complete when given the optimal input.
- It denotes an algorithm operating at its peak efficiency under ideal circumstances.
- **Worst-case complexity ($O(\text{worst})$):** This denotes the maximum time an algorithm will take to finish for any given input.
- It represents the scenario where the algorithm encounters the most unfavourable input.

Time Complexity

- **Average-case complexity ($O(\text{average})$):**
- This estimates the typical running time of an algorithm when averaged over all possible inputs.
- It provides a more realistic evaluation of an algorithm's performance.

Time Complexity

- **Big O Notation**

- In computer science, Big O notation is used more specifically to find the worst case time complexity for an algorithm.
- Big O notation uses a capital letter O with parenthesis(), and inside the parenthesis there is an expression that indicates the algorithm runtime.
- Runtime is usually expressed using n, which is the number of values in the data set the algorithm is working on.

Example 1:

Statement	Step/execution (s/e)	Frequency	Total step
Algorithm sum(a,n)	0	1	0
{	0	1	0
s=0.0;	0	1	0
for i=1 to n do	1	n+1	n+1
s=s+a[i];	1	n	n
return s;	0	1	0
}	0	1	0

Time complexity/ time taken by the above code = $0 + 0 + 0 + n+1 + n + 0 + 0$
 $= 2n + 1$

Example 2:

```
1.int fun1(int n)
2.{
3.  int sum=0;
4.  for(int i=1; i<=n; i++)
5.      for(int j=1; j<=n; j++)
6.          sum++;
7.  return sum;
8.}
```

**Time Complexity = order
of n^2**

and

**Space Complexity = order
of 1 (constant)**

Example 3:

```

1.int sum =0;

2.int fun1(n)

3.{

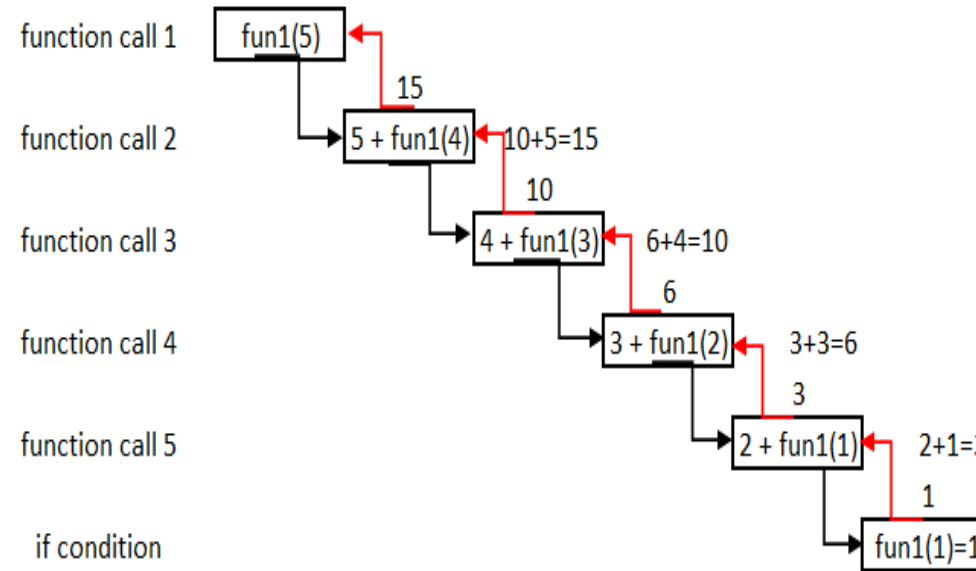
4.  If(n==1)

5.    return 1;

6.  sum = n+ fun1(n-1);

7.  return sum;

8.}
    
```



Time Complexity= order of n

and

Space Complexity= order of n

Example 4:

```
1. int fun2(int n)
2. {
3.     int sum=0;
4.     for(int i=1;i<=n;i++)
5.         sum=sum + i;
6.     return sum;
7. }
```

Time Complexity=
order of n

and

Space Complexity=
order of 1

Example 5:

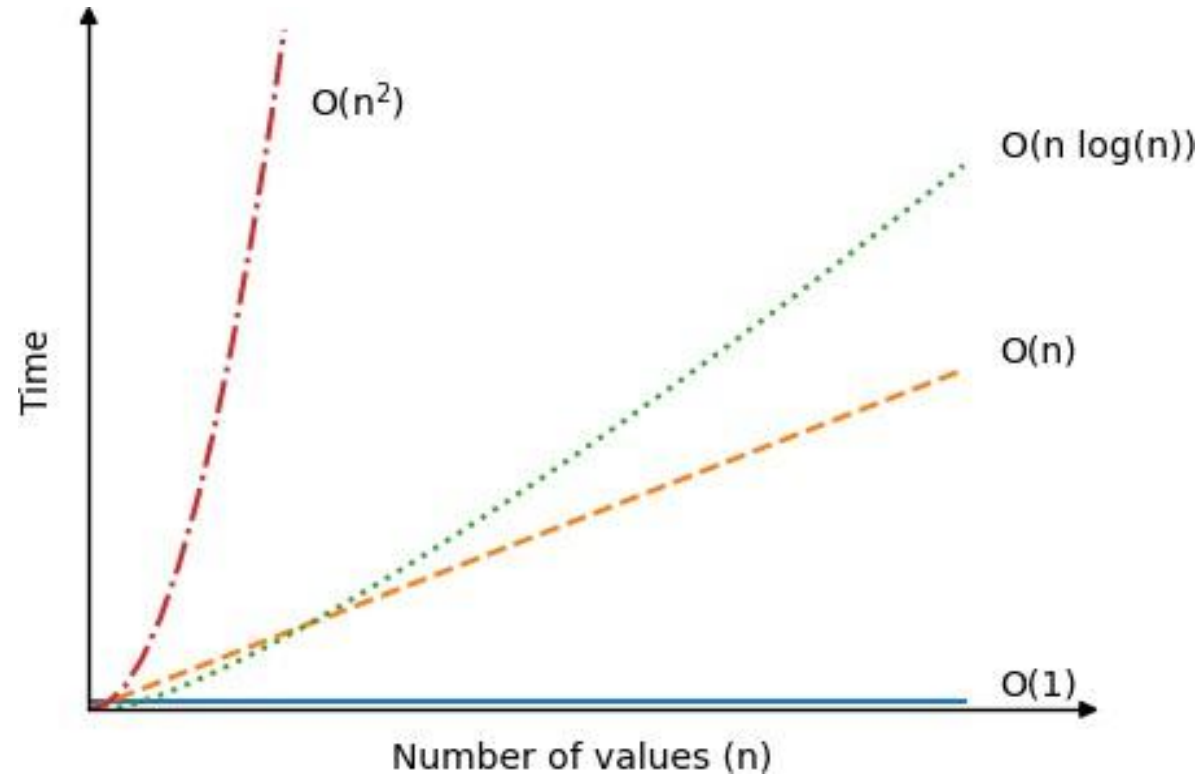
```
1. int fun3(int n)
2. {
3.     Return
   n*(n+1)/2;
4. }
```

Time Complexity=
order of 1

and

Space Complexity=
order of 1

Here is how time increases when the number of values n increase for different algorithms:

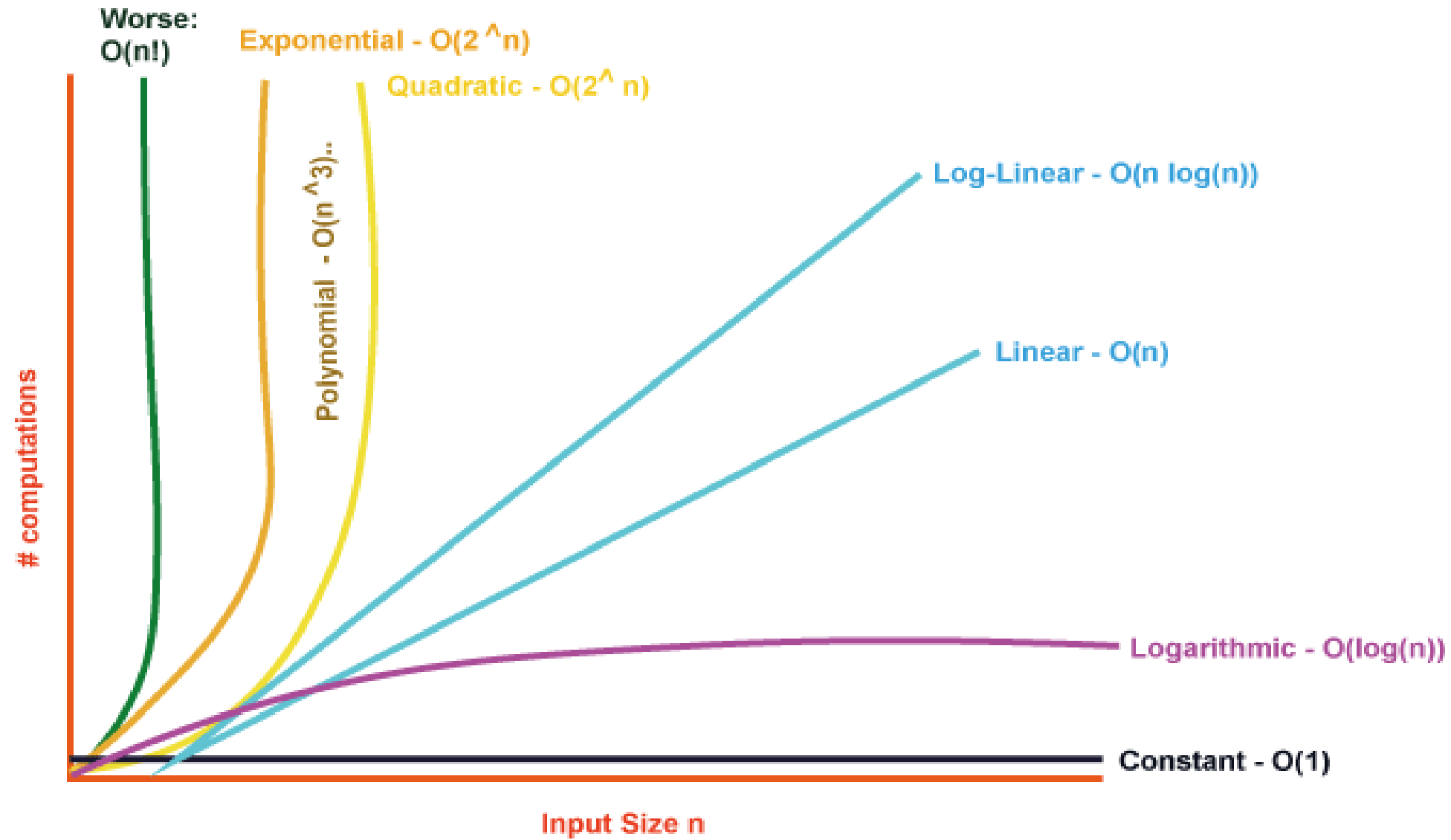




Complexity Summary

Sorting Algorithm	Best Case	Average Case	Worst Case
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$
Insertion Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$
Bubble Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$
Merge Sort	$O(N \cdot \log_2 N)$	$O(N \cdot \log_2 N)$	$O(N \cdot \log_2 N)$
Quick Sort	$O(N \cdot \log_2 N)$	$O(N \cdot \log_2 N)$	$O(N^2)$

Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n)$



Time Complexity

- Conclusion:
- In conclusion, understanding time complexity in data structures is crucial for analyzing and evaluating the efficiency and performance of algorithms.
- Time complexity is a measure of how long an algorithm takes to run as the size of the input increases.
- It provides valuable insights into how an algorithm scales and helps in making informed decisions about algorithm selection and optimization.

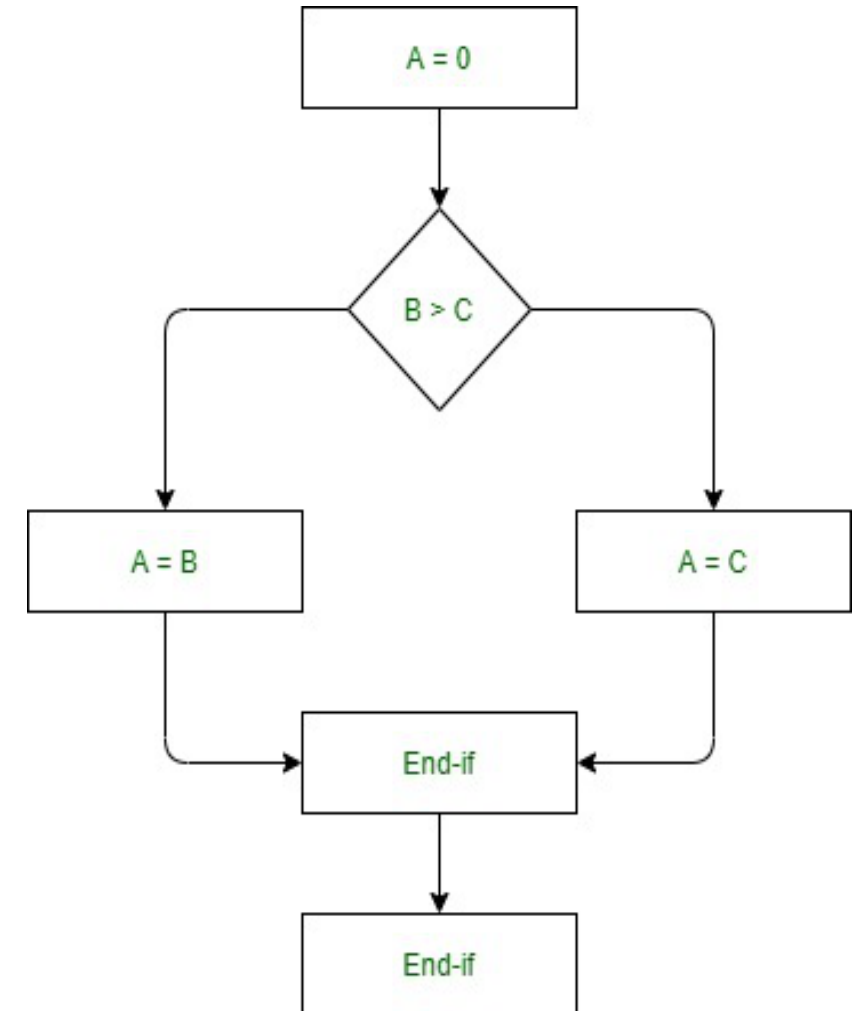
Algorithm Design Techniques

- Iterative techniques
- Divide & Conquer Approach
- Dynamic Programming Approach
- Greedy Approach

- Randomization Approach
- Approximation Approach
- Recursive Approach
- Branch and Bound Approach

Iterative techniques

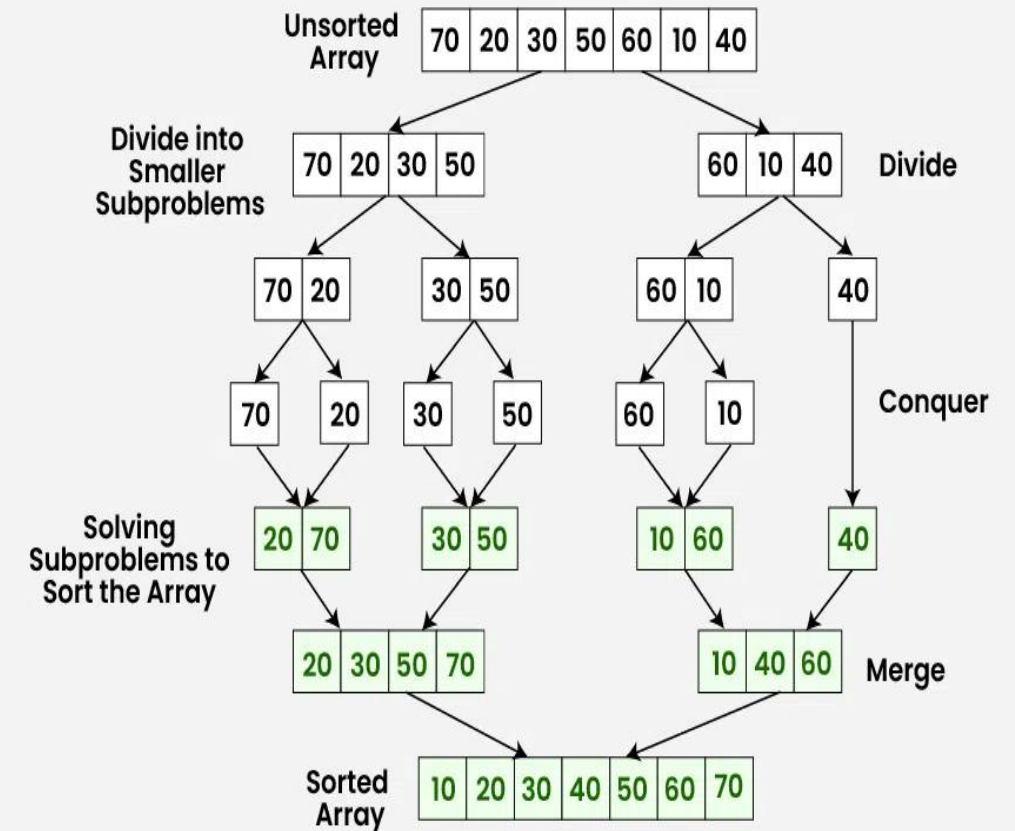
- iterative techniques are essential for building efficient and scalable algorithms that can solve various types of problems through repetition and structured loops.
- The iterative approach involves solving a problem through repeated execution of a set of instructions within a loop (e.g., for, while).
- Common Use Cases:
Traversal, Searching, Sorting, Graph Traversal



Divide & Conquer Approach

- The algorithms which follow the divide & conquer techniques involve three steps:
- Divide the original problem into a set of sub-problems.
- Solve every sub-problem individually, recursively.
- Combine the solution of the sub-problems (top level) into a solution of the whole original problem.
- It is a top-down approach.
- Common Uses:
Merge Sort, Quick Sort, Binary Search.

Working of Divide & Conquer Algorithm

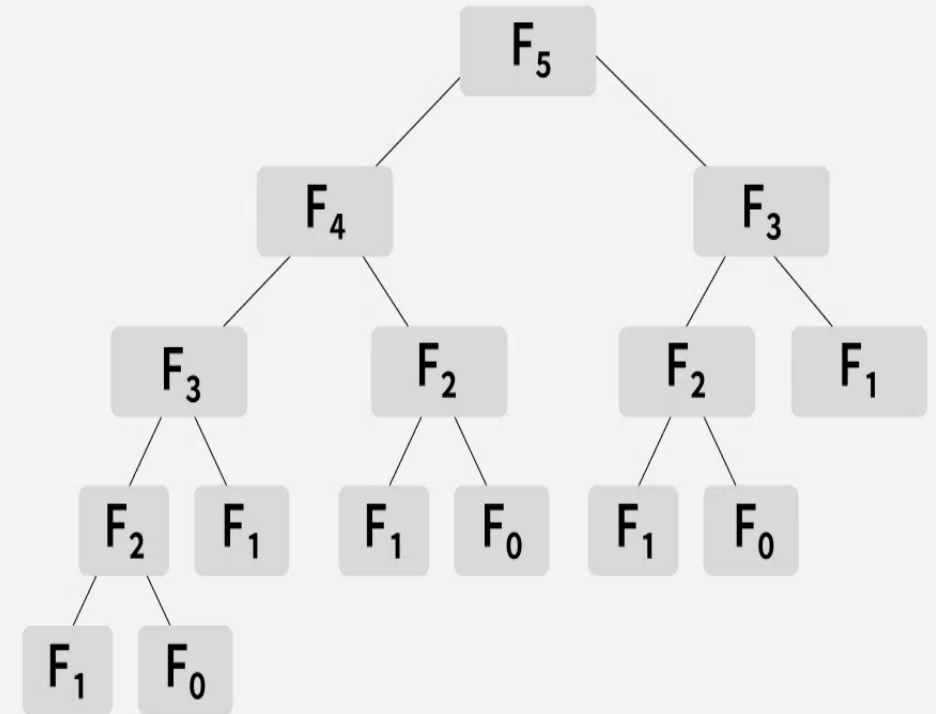


Dynamic Programming Approach

- Dynamic Programming is a bottom-up approach we solve all possible small problems and then combine them to obtain solutions for bigger problems.
- Dynamic Programming is a commonly used algorithmic technique used to optimize recursive solutions when same subproblems are called again.
- Common Uses:
Fibonacci sequence.

N^{th} Term of Fibonacci Series

Here F_n denotes N^{th} Term of the Fibonacci



Greedy Approach

- In the greedy method, at each step, a decision is made to choose the local optimum, without thinking about the future consequences.
- At every step, we can make a choice that looks best at the moment, and we get the optimal solution to the complete problem.
- Some popular Greedy Algorithms are Fractional Knapsack, Dijkstra's algorithm, Kruskal's algorithm, Huffman coding and Prim's Algorithm

